

the essentials of

Computer Organization and Architecture

Linda Null and Julia Lobur

Chapter 1

Introduction

Important Information



- Attending a lecture.
- Arrive on time.
- Don't use mobile phone.
- Not allowed to talk with your friend
- Not allowed to eat or drink
- You can ask any question at any time.
- Negative bonus.

Grading System



- 7th week assessment (30)
- 12th week assessment (20)
- Section Evaluation (10)
- Final Exam (40)

Chapter 1 Objectives



- Know the difference between computer organization and computer architecture.
- Understand units of measure common to computer systems.
- Appreciate the evolution of computers.
- Understand the computer as a layered system.
- Be able to explain the von Neumann architecture and the function of basic computer components.

1.1 Overview



Why study computer organization and architecture?

- Design better programs, including system software such as compilers, operating systems, and device drivers.
- Optimize program behavior.
- Evaluate (benchmark) computer system performance.
- Understand time, space, and price tradeoffs.

1.1 Overview



- Computer organization
 - Encompasses all physical aspects of computer systems.
 - E.g., circuit design, control signals, memory types.
 - *How does a computer work?*
- Computer architecture
 - Logical aspects of system implementation as seen by the programmer.
 - E.g., instruction sets, instruction formats, data types, addressing modes.
 - *How do I design a computer?*

1.2 Computer Components



- There is no clear distinction between matters related to computer organization and matters relevant to computer architecture.
- Principle of Equivalence of Hardware and Software:
 - *Anything that can be done with software can also be done with hardware, and anything that can be done with hardware can also be done with software.**

* Assuming speed is not a concern.

1.2 Computer Components

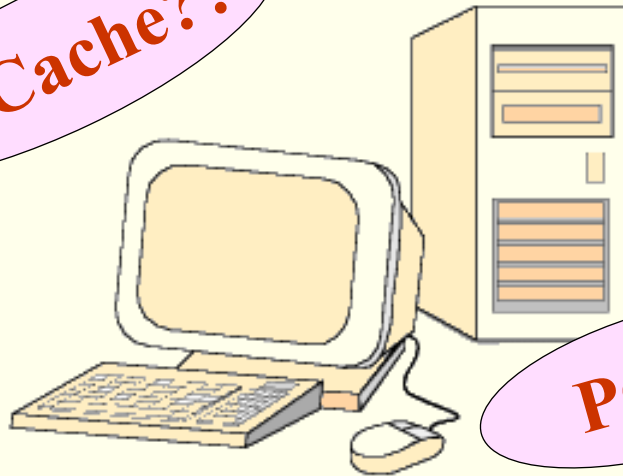


- At the most basic level, a computer is a device consisting of three pieces:
 - A processor to interpret and execute programs
 - A memory to store both data and programs
 - A mechanism for transferring data to and from the outside world.

1.3 An Example System

Consider this advertisement:

For Sale: Obsolete Computer – Cheap! Cheap! Cheap!



PCI??

- Pentium III 667MHz
- 133MHz 64MB SDRAM
- 32KB L1 cache, 256KB L2
- 30GB EIDE hard drive (7200 rpm)
- 48X max variable CD-ROM
- 2 USB ports, 1 serial port, 1 parallel port
- Monitor, 19", .24mm AG, 1280x1024 at 85Hz
- Intel 3D AGP graphics card
- 56K PCI voice modem
- 64-bit PCI sound card

MHz??

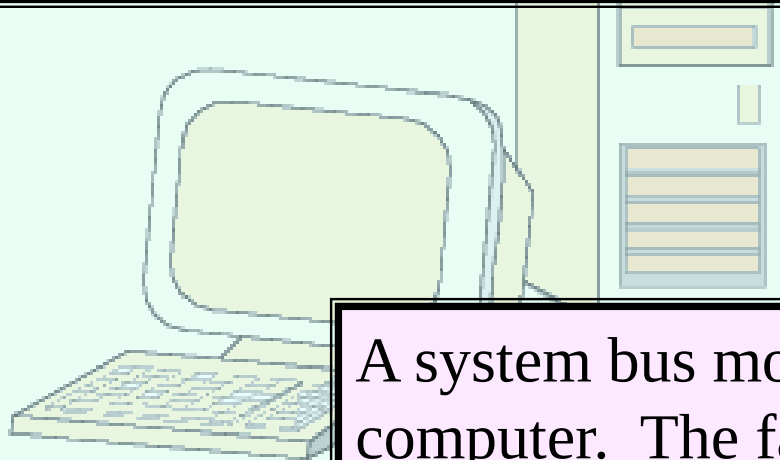
MB??

USB??

What does it all mean??

1.3 An Example System

The microprocessor is the “brain” of the system. It executes program instructions. This one is a Pentium III (Intel) running at 667MHz.



A system bus moves data within the computer. The faster the bus the better. This one runs at 133MHz.

er – Cheap! Cheap! Cheap

- Pentium III 667MHz
- 133MHz 64MB SDRAM
- 32KB L1 cache, 256KB L2 cache
- 50GB EIDE hard drive (7200 FPM)
- 48X max variable CD-ROM
- 2 USB ports, 1 serial port, 1 parallel port
- Monitor, 19", 24mm AG, 1280x1024

1.3 An Example System



- Computers with large main memory capacity can run larger programs with greater speed than computers having small memories.
- RAM is an acronym for *random access memory*. Random access means that memory contents can be accessed directly if you know its location.
- Cache is a type of temporary memory that can be accessed faster than RAM.

1.3 An Example System

Throughout the remainder of this book you will see how these components work and how they interact with software to make complete computer systems.

This statement raises two important questions:

What assurance do we have that computer components will operate as we expect?

And what assurance do we have that computer components will operate together?

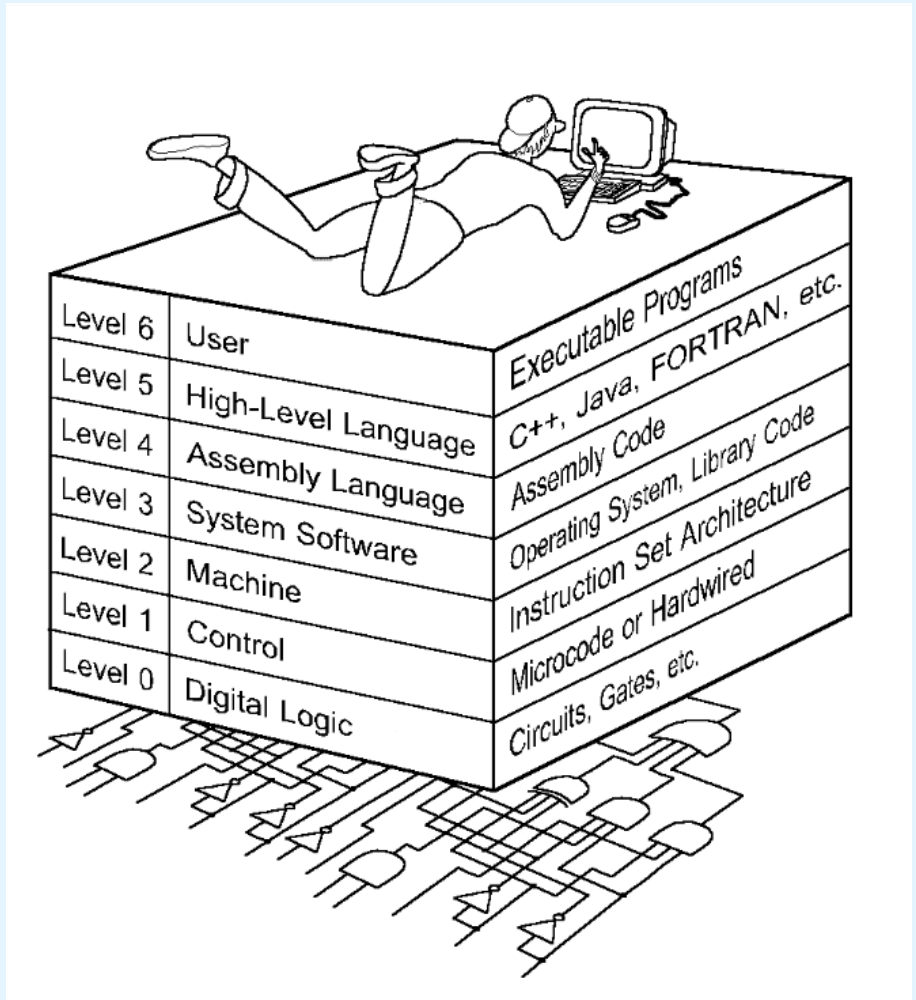
1.6 The Computer Level Hierarchy



- Computers consist of many things besides chips.
- Before a computer can do anything worthwhile, it must also use software.
- Writing complex programs requires a “divide and conquer” approach, where each program module solves a smaller problem.
- Complex computer systems employ a similar technique through a series of virtual machine layers.

1.6 The Computer Level Hierarchy

- Each virtual machine layer is an abstraction of the level below it.
- The machines at each level execute their own particular instructions, calling upon machines at lower levels to perform tasks as required.
- Computer circuits ultimately carry out the work.



1.6 The Computer Level Hierarchy



- Level 6: The User Level
 - Program execution and user interface level.
 - The level with which we are most familiar.
- Level 5: High-Level Language Level
 - The level with which we interact when we write programs in languages such as C, Pascal, Lisp, and Java.

1.6 The Computer Level Hierarchy



- Level 4: Assembly Language Level
 - Acts upon assembly language produced from Level 5, as well as instructions programmed directly at this level.
- Level 3: System Software Level
 - Controls executing processes on the system.
 - Protects system resources.
 - Assembly language instructions often pass through Level 3 without modification.

1.6 The Computer Level Hierarchy



- Level 2: Machine Level
 - Also known as the Instruction Set Architecture (ISA) Level.
 - Consists of instructions that are particular to the architecture of the machine.
 - Programs written in machine language need no compilers, interpreters, or assemblers.

1.6 The Computer Level Hierarchy



- Level 1: Control Level
 - A *control unit* decodes and executes instructions and moves data through the system.
 - Control units can be *microprogrammed* or *hardwired*.
 - A microprogram is a program written in a low-level language that is implemented by the hardware.
 - Hardwired control units consist of hardware that directly executes machine instructions.

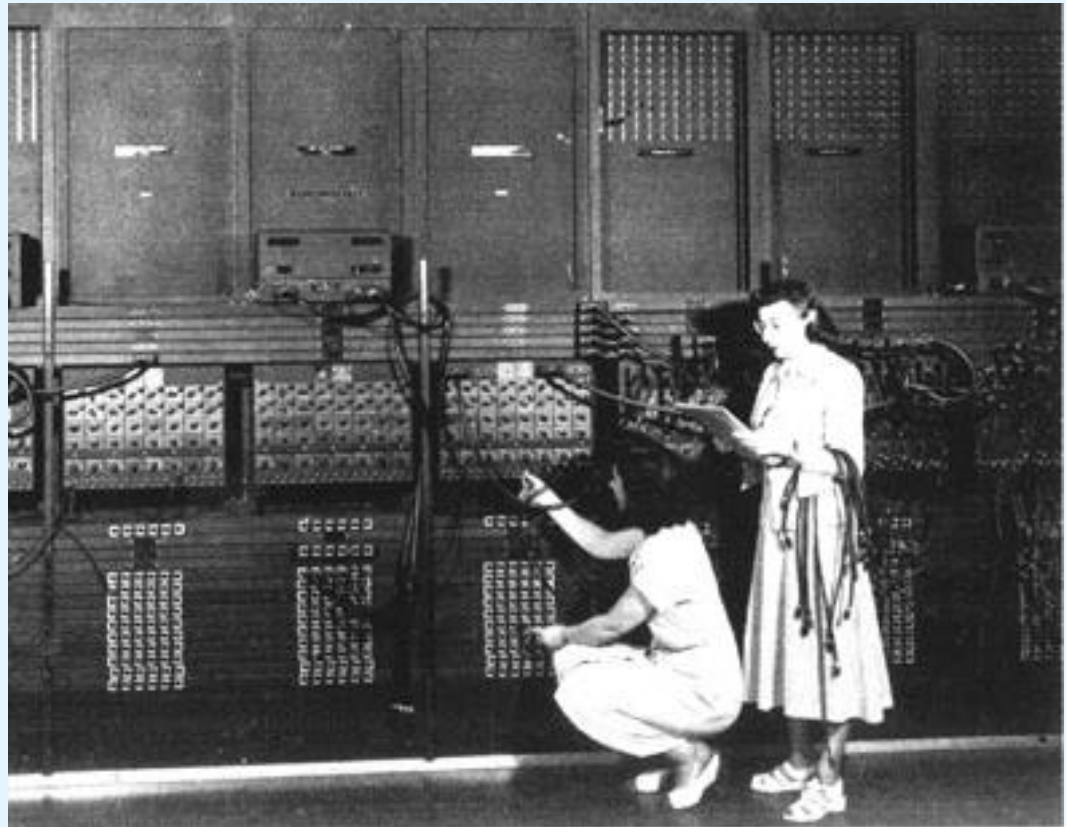
1.6 The Computer Level Hierarchy



- Level 0: Digital Logic Level
 - This level is where we find digital circuits (the chips).
 - Digital circuits consist of gates and wires.
 - These components implement the mathematical logic of all other levels.

1.7 The von Neumann Model

- On the ENIAC, all programming was done at the digital logic level.
- Programming the computer involved moving plugs and wires.



1.7 The von Neumann Model



- Inventors of the ENIAC, John Mauchley and J. Presper Eckert, conceived of a computer that could store instructions in memory.
- The invention of this idea has since been ascribed to a mathematician, John von Neumann, who was a contemporary of Mauchley and Eckert.
- Stored-program computers have become known as von Neumann Architecture systems.

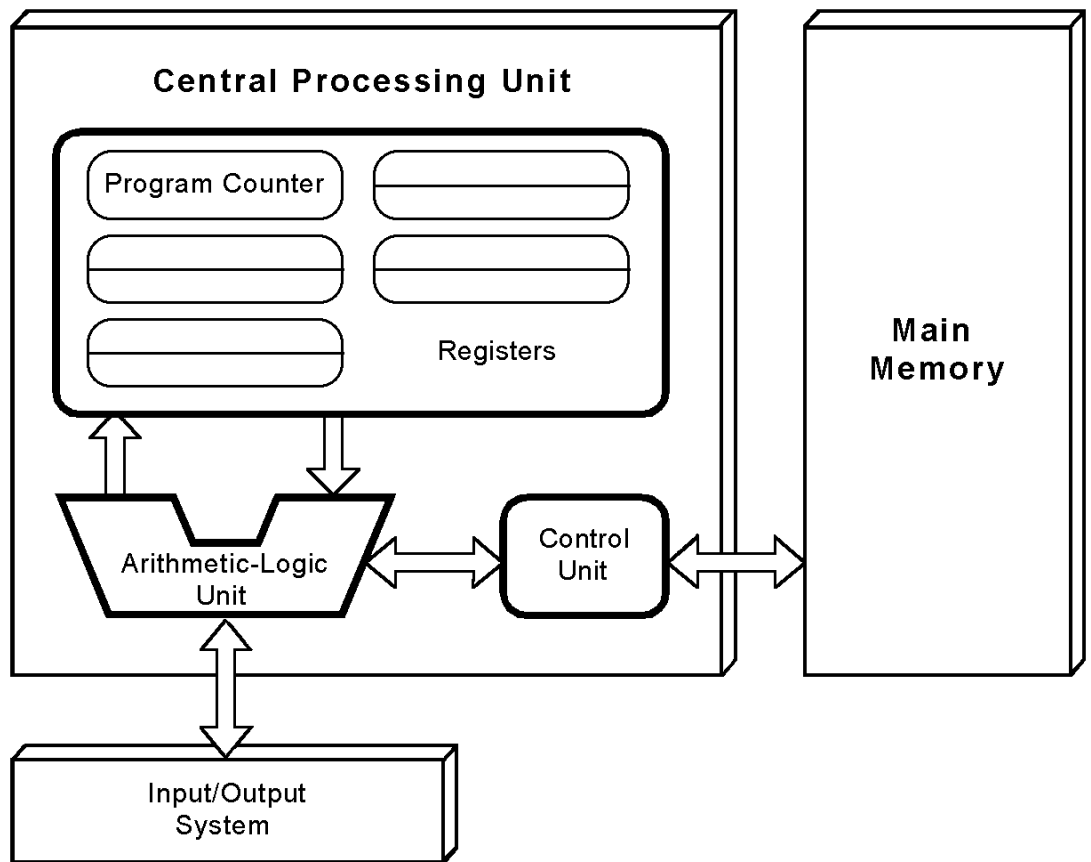
1.7 The von Neumann Model



- Today's stored-program computers have the following characteristics:
 - Three hardware systems:
 - A central processing unit (CPU)
 - A main memory system
 - An I/O system
 - The capacity to carry out sequential instruction processing.
 - A single data path between the CPU and main memory.
 - This single path is known as the *von Neumann bottleneck*.

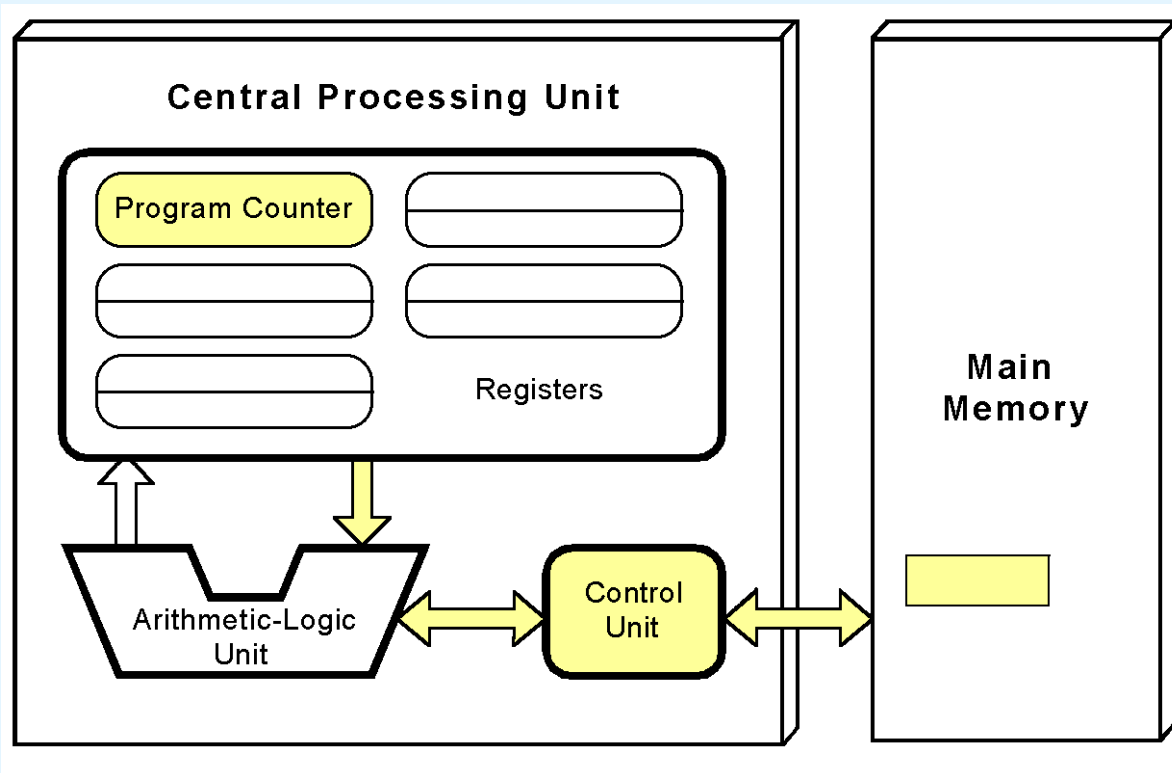
1.7 The von Neumann Model

- This is a general depiction of a von Neumann system:
- These computers employ a fetch-decode-execute cycle to run programs as follows . . .



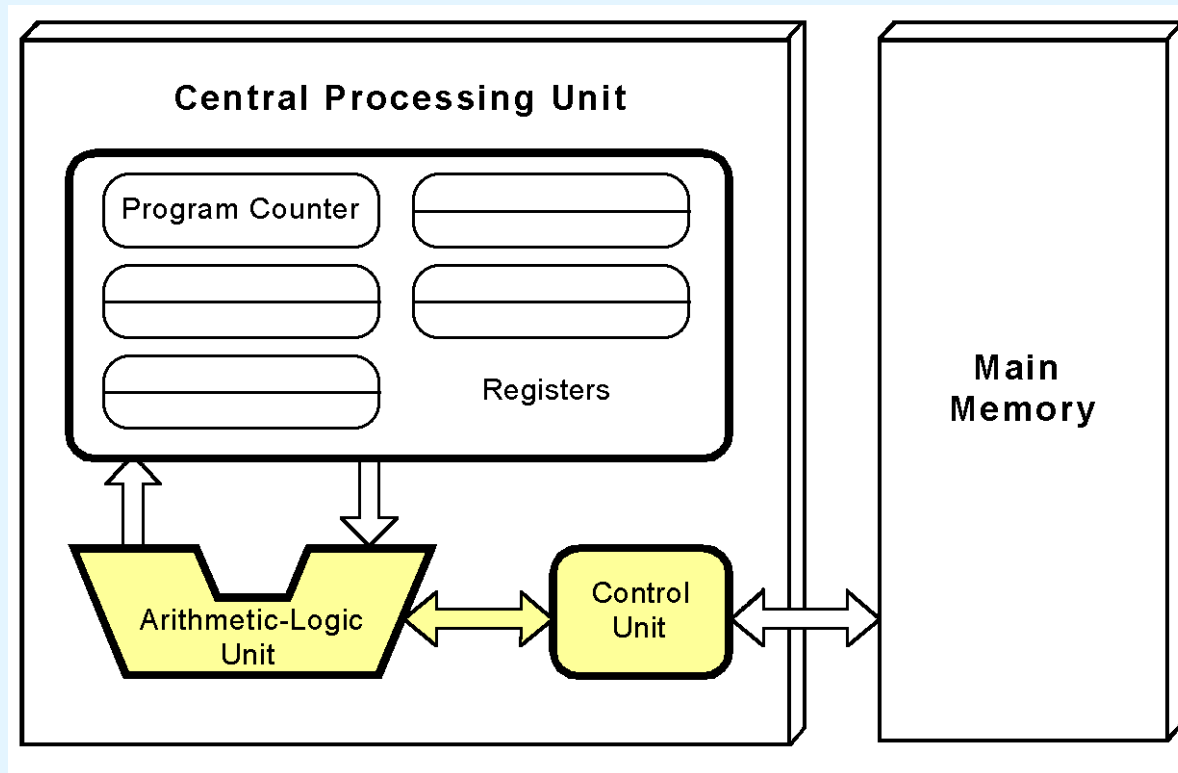
1.7 The von Neumann Model

- The control unit fetches the next instruction from memory using the program counter to determine where the instruction is located.



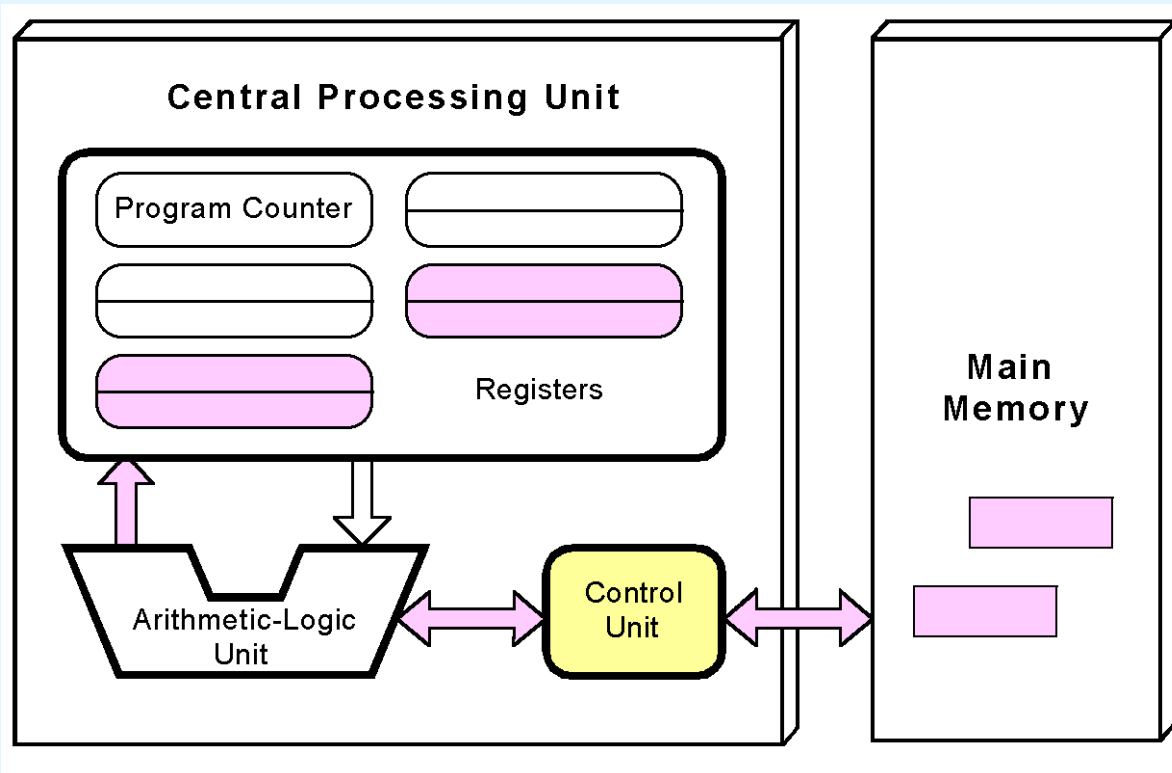
1.7 The von Neumann Model

- The instruction is decoded into a language that the ALU can understand.



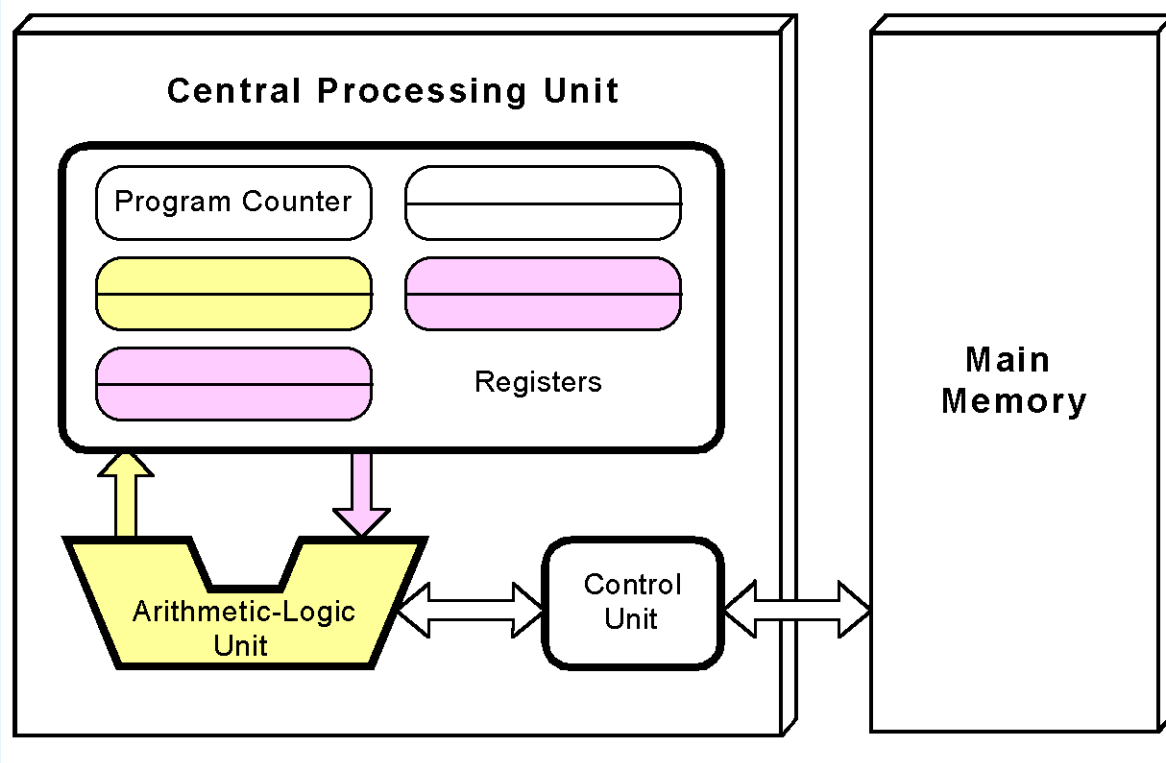
1.7 The von Neumann Model

- Any data operands required to execute the instruction are fetched from memory and placed into registers within the CPU.



1.7 The von Neumann Model

- The ALU executes the instruction and places results in registers or memory.



1.8 Non-von Neumann Models



- Conventional stored-program computers have undergone many incremental improvements over the years.
- These improvements include adding specialized buses, floating-point units, and cache memories, to name only a few.
- But enormous improvements in computational power require departure from the classic von Neumann architecture.
- Adding processors is one approach.

1.8 Non-von Neumann Models



- In the late 1960s, high-performance computer systems were equipped with dual processors to increase computational throughput.
- In the 1970s supercomputer systems were introduced with 32 processors.
- Supercomputers with 1,000 processors were built in the 1980s.
- In 1999, IBM announced its Blue Gene system containing over 1 million processors.

1.8 Non-von Neumann Models



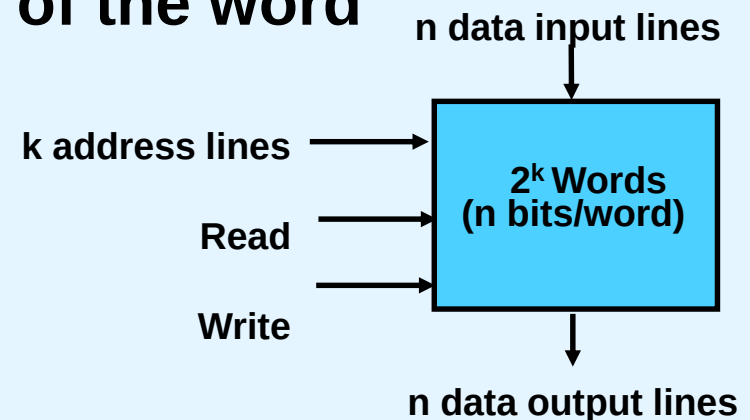
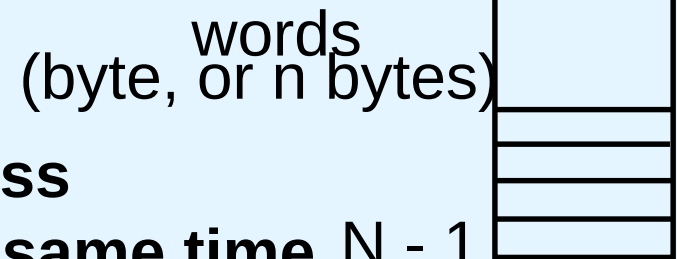
- Parallel processing is only one method of providing increased computational power.
- More radical systems have reinvented the fundamental concepts of computation.
- These advanced systems include genetic computers, quantum computers, and dataflow systems.
- At this point, it is unclear whether any of these systems will provide the basis for the next generation of computers.

MEMORY COMPONENTS

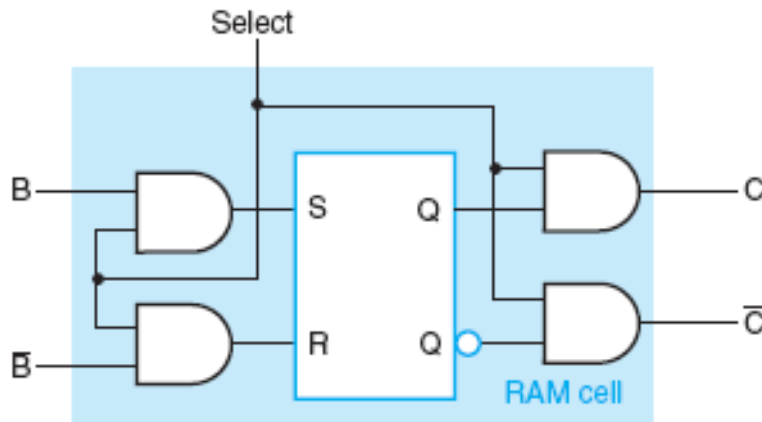
Logical Organization

Random Access Memory

- Each word has a unique address
- Access to a word requires the same time independent of the location of the word
- Organization



MEMORY COMPONENTS



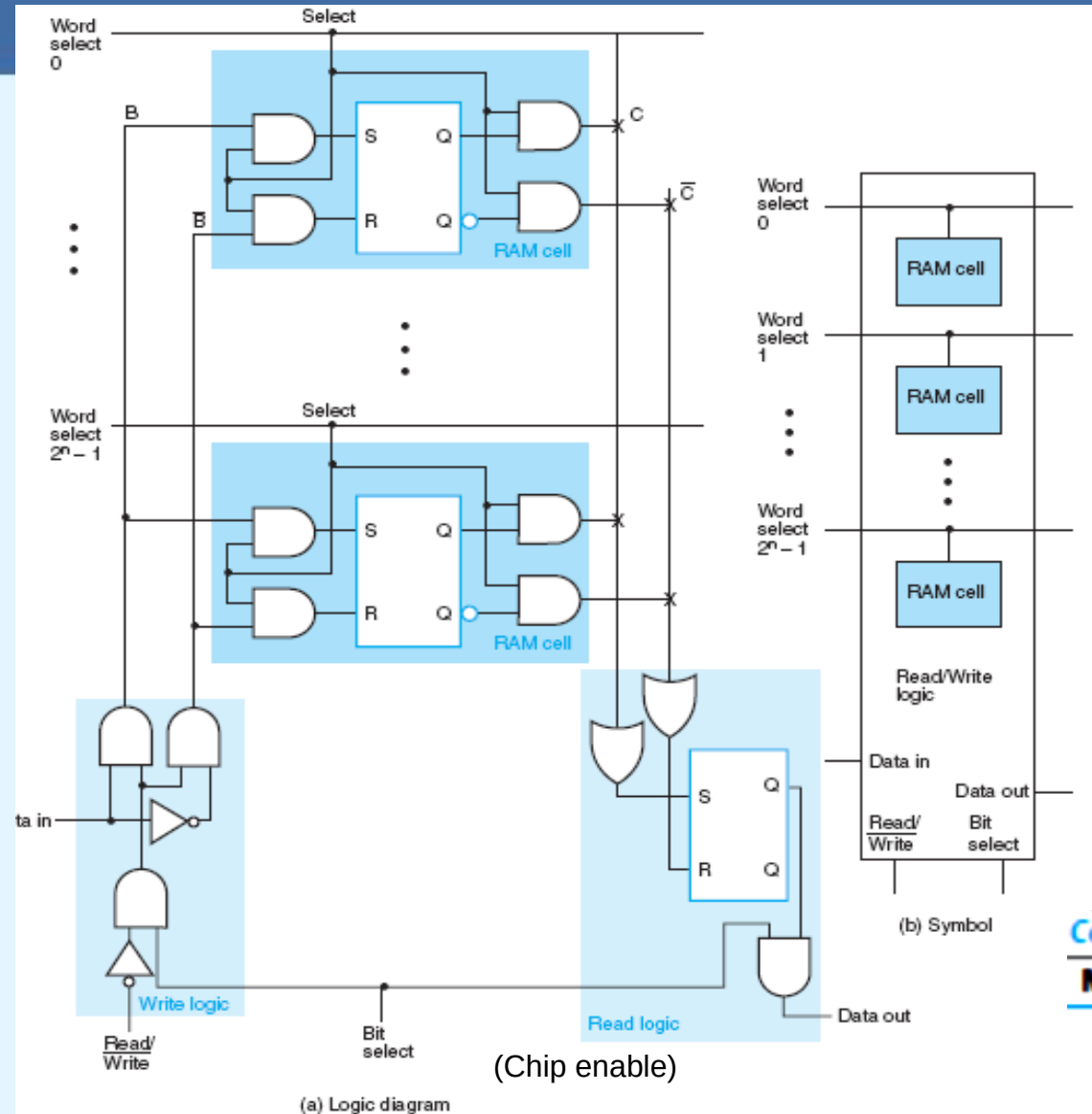
Static RAM Cell

Select = 0 (cell disable)
Select = 1 (cell enable)

Memory address		Memory contents
Binary	Decimal	
0000000000	0	10110101 01011100
0000000001	1	10101011 10001001
0000000010	2	00001101 01000110
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
1111111101	1021	10011101 00010101
1111111110	1022	00001101 00011110
1111111111	1023	11011110 00100100

Contents of a 1024×16 Memory

MEMORY COMPONENTS

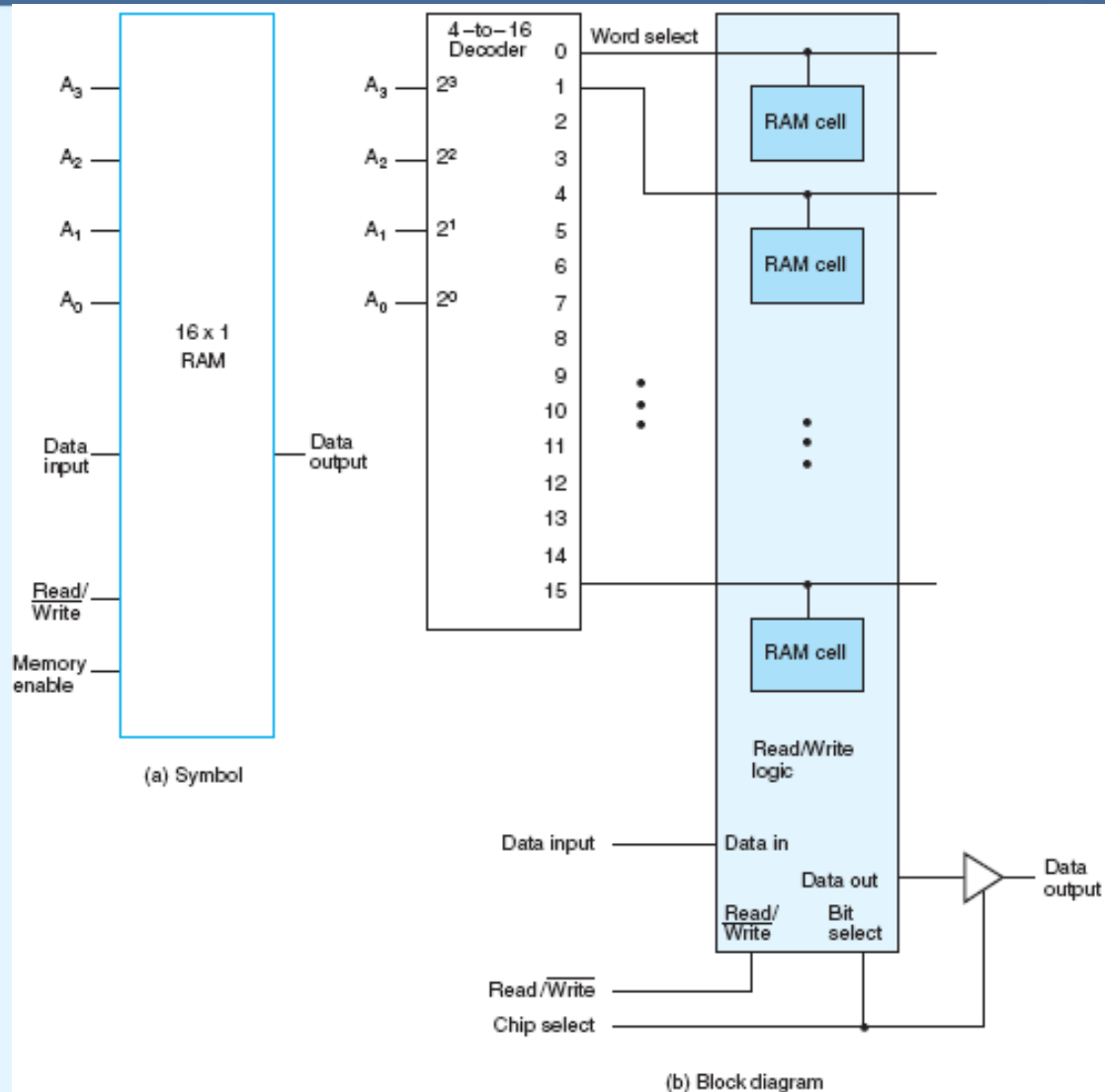


RAM Bit Slice Model

Control Inputs to Memory Chip

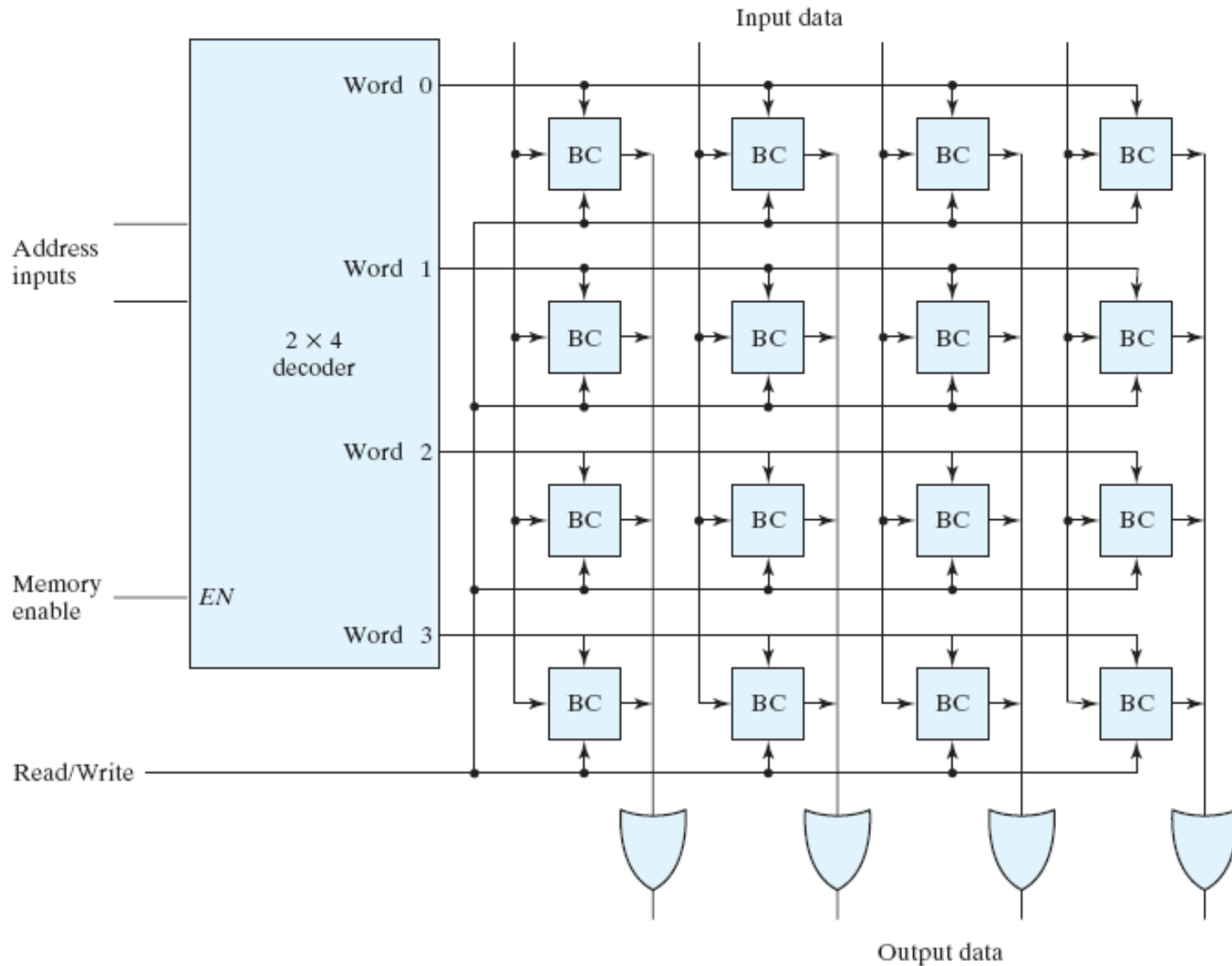
Memory Enable	Read/Write	Memory Operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

MEMORY COMPONENTS



16-Word by 1-Bit RAM Chip

MEMORY COMPONENTS



Random Access Memory (RAM)

- It is also called read-write *memory* or the *main memory* or the *primary memory*.
- The programs and data that the CPU requires during the execution of a program are stored in this memory.
- It is a volatile memory as the data is lost when the power is turned off.
- RAM is further classified into two types:
 - SRAM (*Static Random Access Memory*).
 - DRAM (*Dynamic Random Access Memory*).

Random Access Memory (RAM)

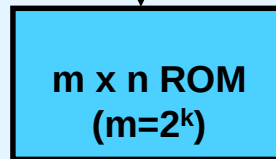
DRAM	SRAM
1. Constructed of tiny capacitors that leak electricity.	1. Constructed of circuits similar to D flip-flops.
2. Requires a recharge every few milliseconds to maintain its data.	2. Holds its contents as long as power is available.
3. Inexpensive.	3. Expensive.
4. Slower than SRAM.	4. Faster than DRAM.
5. Can store many bits per chip.	5. Can not store many bits per chip.
6. Uses less power.	6. Uses more power.
7. Generates less heat.	7. Generates more heat.
8. Used for main memory.	8. Used for cache.

READ ONLY MEMORY(ROM)

Characteristics

- Perform read operation only,
- write operation is not possible
- Information stored in a ROM is made permanent during production, and cannot be changed
- Organization

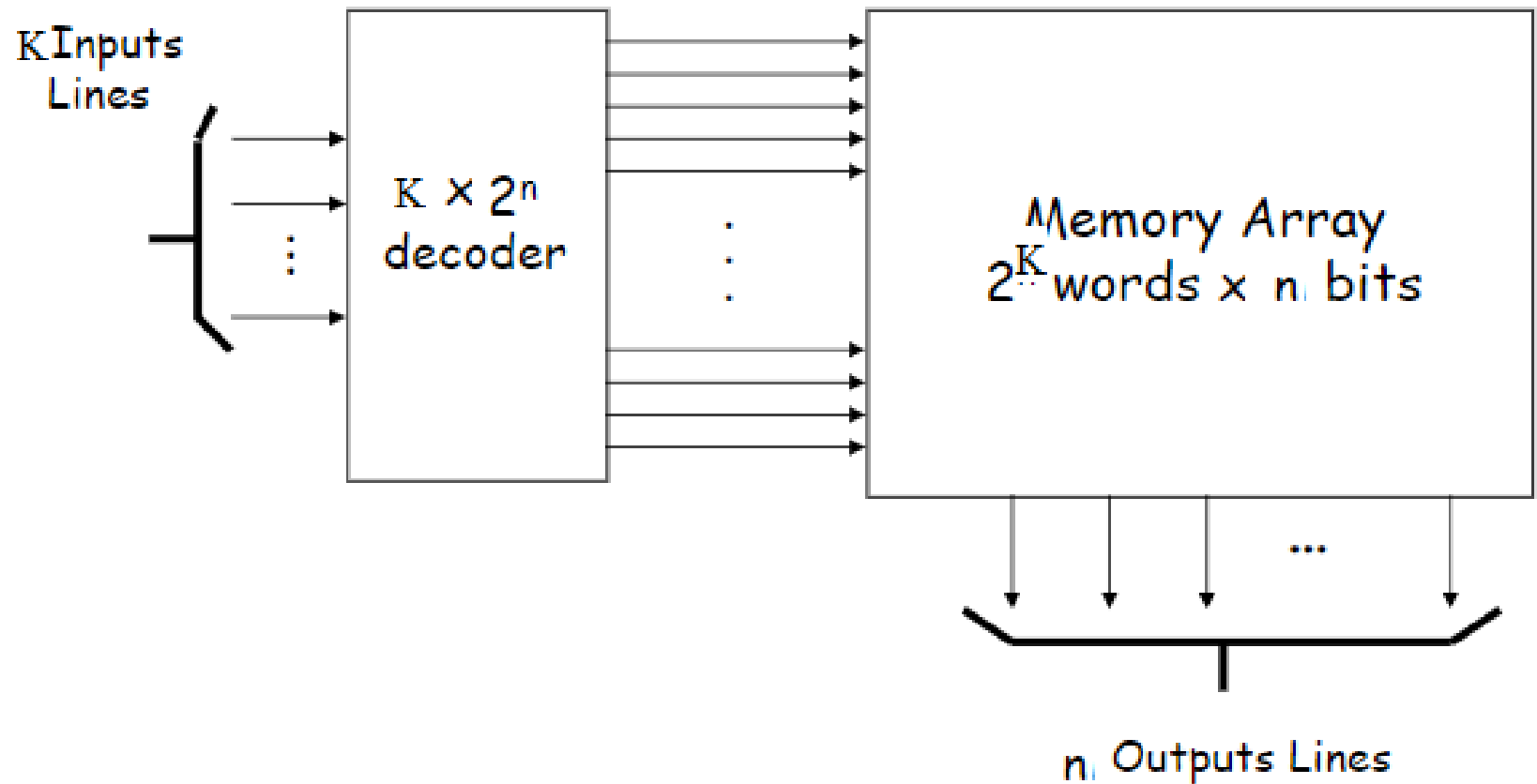
k address input lines



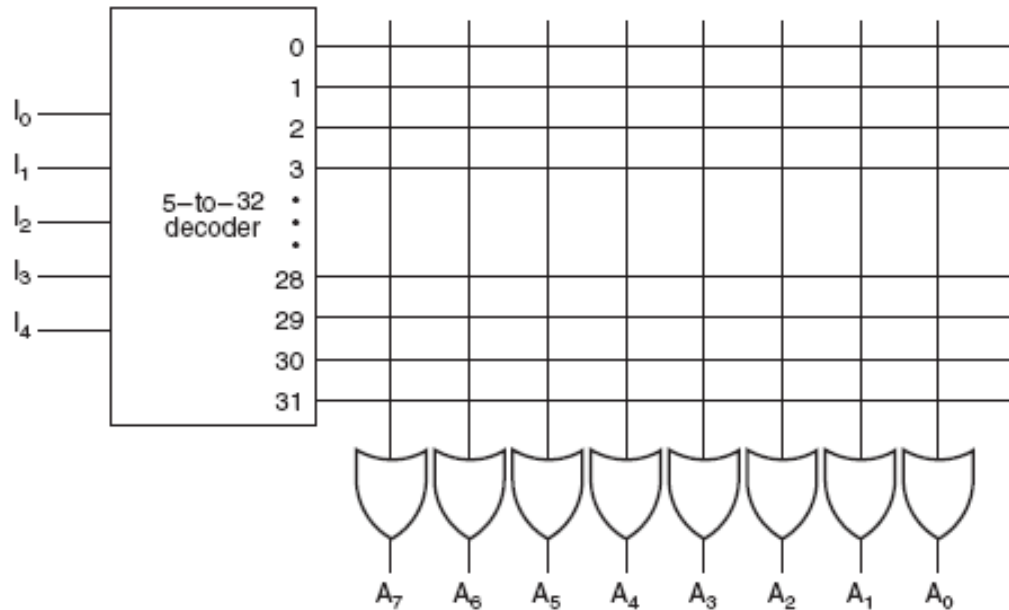
n data output lines

- Information on the data output line depends only on the information on the address input lines.

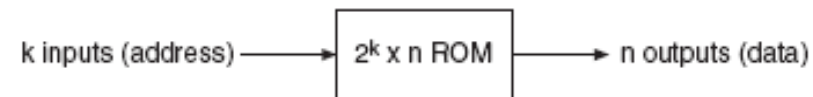
READ ONLY MEMORY(ROM)



READ ONLY MEMORY(ROM)



Internal Logic of a 32×8 ROM



Block Diagram of ROM

READ ONLY MEMORY(ROM)

Specify a truth table for a ROM which implements:

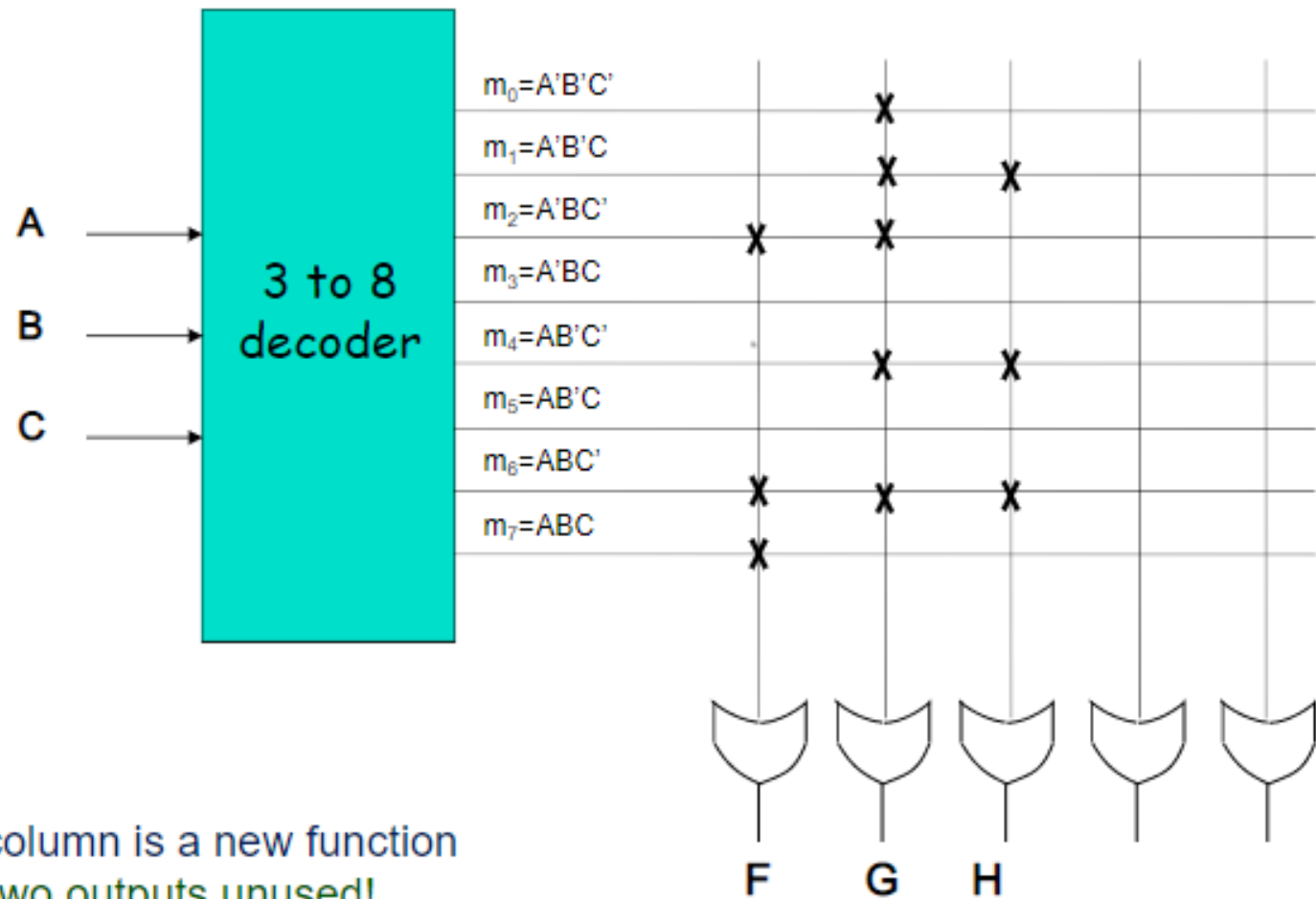
$$F = AB + A'BC'$$

$$G = A'B'C + C'$$

$$H = AB'C' + ABC' + A'B'C$$

A	B	C	F	G	H	
0	0	0	0	1	0	m_0
0	0	1	0	1	1	m_1
0	1	0	1	1	0	m_2
0	1	1	0	0	0	m_3
1	0	0	0	1	1	m_4
1	0	1	0	0	0	m_5
1	1	0	1	1	1	m_6
1	1	1	1	0	0	m_7

READ ONLY MEMORY(ROM)



TYPES OF ROM

ROM

- Store information (function) during production
- Mask is used in the production process
- Unalterable
- Low cost for large quantity production, used in the final products

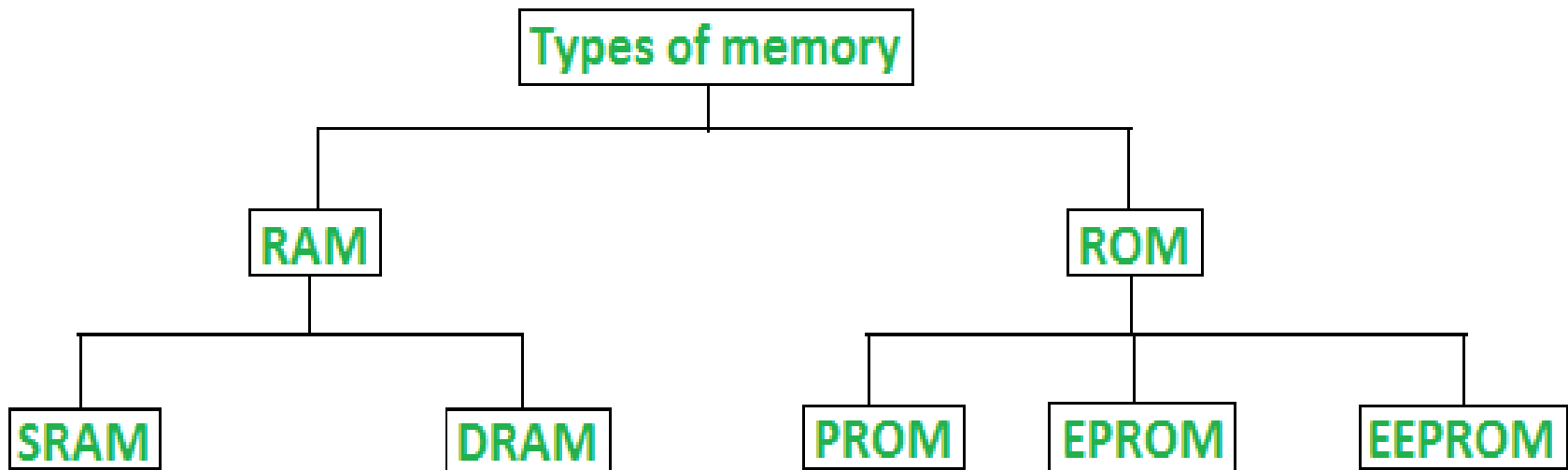
PROM (Programmable ROM)

- Store info electrically using PROM programmer at the user's site
- Unalterable
- Higher cost than ROM -> used in the system development phase
 - > Can be used in small quantity system

EPROM (Erasable PROM)

- Store info electrically using PROM programmer at the user's site
- Stored info is erasable (alterable) using UV light (electrically in some devices) and rewriteable
- Higher cost than PROM but reusable --> used in the system development phase. Not used in the system production due to reusability

Types of Memory



Classification of computer memory

Difference between RAM and ROM

RAM	ROM
1. Temporary Storage.	1. Permanent storage.
2. Store data in MBs.	2. Store data in GBs.
3. Volatile.	3. Non-volatile.
4.Used in normal operations.	4. Used for startup process of computer.
5. Writing data is faster.	5. Writing data is slower.

Difference between RAM and ROM

Conclusion



- This chapter has given you an overview of the subject of computer architecture.
- You should now be sufficiently familiar with general system structure to guide your studies throughout the remainder of this course.
- Subsequent chapters will explore many of these topics in great detail.